SPRINT 4 · PYTHON PARA IA · S04.2

Estructuras de datos, ficheros y errores

Una reserva es un diccionario; un fichero de reservas, una lista de diccionarios. Con eso y un poco de cuidado se construye `leer_reservas`.

Listas, tuplas, diccionarios y conjuntos

Lista `[ ]`

Ordenada y modificable. Cortes: `noches_semana[-3:]` (Ejercicio 1: fin de semana y total).

Tupla `( )`

Ordenada e **inmutable**: coordenadas, pares fijos.

Diccionario `{ }`

Clave → valor. Una reserva: `{"canal": "directo", "noches": 3}`. Contar: `conteo[r] = conteo.get(r, 0) + 1`.

Conjunto `{ }`

Sin repetidos: `len(set(canales))` (Ejercicio 4).

Una lista de diccionarios es una tabla

```
reservas = [
    {"canal": "directo", "noches": 3, "precio_noche": 80.0, "cancelada": 0},
    {"canal": "agencia", "noches": 5, "precio_noche": 95.0, "cancelada": 1},
]
# Ejercicio 3: comprensión con filtro
ingresos_confirmados = [r["noches"] * r["precio_noche"]
                        for r in reservas if r["cancelada"] == 0]
```

Mini-reto del cuaderno S04_03: `tasa_por_canal` acumulando en dos diccionarios (`total` y `canceladas`). Resultado esperado: directo 0.0, web_intermediaria 100.0, agencia 50.0.

Leer CSV y JSON con la biblioteca estándar

```
import csv, json
with open("reservas_turisdata.csv", newline="", encoding="utf8") as f:
    filas = list(csv.DictReader(f))      # todo llega como TEXTO
noches = int(filas[0]["noches"])         # hay que convertir
json.dump(kpis, open("kpis.json", "w"), ensure_ascii=False, indent=2)
```

- `DictReader` da cada fila como diccionario; `with` cierra el archivo solo.
- Ejercicio 2 del cuaderno S04_04: ingreso medio de las **no canceladas** (convierte `noches` a `int` y `precio_noche` a `float`).

Cuando algo falla: excepciones

```
def a_float_seguro(texto, defecto=None):
    try:
        return float(texto)
    except (ValueError, TypeError):      # nunca un except: vacío
        return defecto

raise FileNotFoundError(f"No encuentro el fichero: {ruta}")
```

- `FileNotFoundError`, `ValueError`, `KeyError`: capturamos **ese** tipo concreto.
- `else` si no hubo error; `finally` se ejecuta siempre.
- Ejercicio 5: `leer_csv(ruta, columnas_obligatorias)` avisa con un mensaje que **nombra** las columnas que faltan.

> Descartar filas malas sin contarlas es peligroso: el mini-reto devuelve `filas_descartadas`.

Del cuaderno al reto: `kpis.py`

Función de la plantilla	Qué hace	Cuaderno que la prepara
`leer_reservas(ruta)`	Lee, valida y devuelve `(reservas, avisos)`	S04_04
`tasa_cancelacion_por_canal`	% de canceladas por canal (1 decimal)	S04_03
`ingreso_medio_por_reserva`	Media de `noches × precio_noche` de las no canceladas	S04_02
`ocupacion_proxy`	Ocupación **aproximada** con la capacidad	S04_01
`exportar_json`	Guarda el resultado con tildes y sangría	S04_04

Orden de trabajo: ver las pruebas **fallar** → rellenar una función → ver su prueba **pasar** → *commit*.

Buenas prácticas y POO (para casa)

- **PEP 8**: `nombres_con_guion_bajo`, 4 espacios, *docstring* en cada función.
- Funciones pequeñas: leer, calcular y exportar son **funciones distintas**.
- Commits con mensaje útil: «Valida columnas obligatorias en leer_reservas».
- POO (cuaderno **S04_05**, ampliación): `Reserva` con `ingreso()`, `@classmethod desde_fila`, herencia `ReservaGrupo` (10 % de descuento) y `ColeccionKPI`.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

RETO

Reto del sprint — 130 min

Preparar la carpeta y ver que las pruebas fallan → rellenar las funciones una a una → exportar y ejecutar el…

sprints/S04/guia/Reto_Sprint_04.md